

code_dont_fail_me_now

Shahjalal University of Science and Technology

1	Geometry	1
1.1	Full Template	1
2	Geometry	1
2.1	Geometry Primitives	1
2.2	Point 2D	1
2.3	Line Intersection	1
2.4	Line Equation	1
2.5	Segment Distance	1
2.6	Segment Intersection	1
2.7	Side Of	1
2.8	On Segment	1
2.9	Linear Transformation	2
2.10	Angle class	2
2.11	Circle Tangents	2
2.12	Minimum Enclosing Circle	2
2.13	Inside Polygon	2
2.14	Polygon Area	2
2.15	Polygon Center	2
2.16	Convex Hull	2
2.17	Hull Diameter	2
2.18	Point Inside Hull	2
2.19	Line Hull Intersection	3
2.20	Closest Pair	3
2.21	Polyhedron Volume	3
2.22	Point 3D	3
2.23	3D Convex Hull	3
2.24	Spherical Distance	3

1 Geometry

1.1 Full Template

2 Geometry

2.1 Geometry Primitives

```
#define EPS 1e-12
#define pi 3.1415926535897931159979634\
68544185161590576171875L
bool feq(const long double& a, const
long double& b) {
    long double del = fabs1(a - b);
    if (del <= EPS) return true;
    return del <= EPS * max(fabs1(a),
        fabs1(b));
}
```

2.2 Point 2D

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T>
int sgn(T x) {
    return (x > 0) - (x < 0);
}
template <class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
    bool operator<(P p) const { return tie
        (x, y) < tie(p.x, p.y); }
    bool operator==(P p) const { return
        tie(x, y) == tie(p.x, p.y); }
    P operator+(P p) const { return P(x +
        p.x, y + p.y); }
    P operator-(P p) const { return P(x -
        p.x, y - p.y); }
    P operator*(T d) const { return P(x *
        d, y * d); }
    P operator/(T d) const { return P(x /
        d, y / d); }
```

```
T dot(P p) const { return x * p.x + y
    * p.y; }
T cross(P p) const { return x * p.y -
    y * p.x; }
T cross(P a, P b) const { return (a -
    *this).cross(b - *this); }
T dist2() const { return x * x + y * y
    ; }
double dist() const { return sqrt((
    double)dist2()); }
// angle to x-axis in interval [-pi,
    pi]
double angle() const { return atan2(y,
    x); }
P unit() const { return *this / dist()
    ; } // makes dist()=1
P perp() const { return P(-y, x); }
// rotates +90 degrees
P normal() const { return perp().unit
    (); }
// returns point rotated 'a' radians
    ccw around the origin
P rotate(double a) const {
    return P(x * cos(a) - y * sin(a), x
        * sin(a) + y * cos(a));
}
friend ostream& operator<<(ostream& os
    , P p) {
    return os << "(" << p.x << ", " << p.
        y << ")";
}
};
```

2.3 Line Intersection

Description: If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point||l and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage:

```
auto res = lineInter(s1,e1,s2,e2); if
(res.first == 1) cout << "intersection point
at " << res.second << endl;
```

```
template <class P>
pair<int, P> lineInter(P s1, P e1, P s2,
    P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P
            (0, 0)};
    auto p = s2.cross(e1, e2), q = s2.
        cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

2.4 Line Equation

```
array<T, 3> line_equation(const Point<T
    >&b, const Point<T>&c){
    array<T, 3>res;
    res[0] = b.y - c.y, res[1] = -(b.x - c.
        x), res[2] = b.x*c.y - b.y*c.x;
    if(res[0] < 0)res[0] = -res[0], res[1]
        = -res[1], res[2] = -res[2];
    else if(res[0] == 0 and res[1] < 0)res
        [1] = -res[1], res[2] = -res[2]; //
        for double use feq(res[0], 0)
    return res;
}
```

2.5 Segment Distance

Description: Returns the shortest distance between point p and the line segment from point s to e.

Usage:

```
Point<double> a, b(2,2), p(1,1); bool
onSegment = segDist(a,b,p) < 1e-10;
```

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s == e) return (p - s).dist();
    auto d = (e - s).dist2(), t = min(d,
        max(0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).
        dist() / d;
}
```

2.6 Segment Intersection

Description: If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point||l and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage:

```
vector<P> inter = segInter(s1,e1,s2,e2); if
(sz(inter)==1) cout << "segments intersect at
" << inter[0] << endl;
```

```
template <class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(
        d, b), oc = a.cross(b, c),
        od = a.cross(b, d);
    // Checks if intersection is single
        non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) *
        sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa
            )};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

2.7 Side Of

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point||T where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage:

```
bool left = sideOf(p1,p2,q)==1;
```

```
template <class P>
int sideOf(P& s, P& e, P& p) {
    return sgn(s.cross(e, p));
}
template <class P>
int sideOf(const P& s, const P& e, const
    P& p, double eps) {
    auto a = (e - s).cross(p - s);
    double l = (e - s).dist() * eps;
    return (a > l) - (a < -l);
}
```

2.8 On Segment

Description: Returns true iff p lies on the line segment from s to e.

Usage:

```
\texttt{ (segDist(s,e,p)<=epsilon)} instead
when using Point<double>.
```

```
template <class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).
        dot(e - p) <= 0;
}
```

2.9 Linear Transformation

Description: Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
typedef Point<double> P;
P linearTransformation(const P& p0,
    const P& p1, const P& q0, const P&
    q1,
    const P& r) {
    P dp = p1 - p0, dq = q1 - q0, num(dp.
        cross(dq), dp.dot(dq));
    return q0 + P((r - p0).cross(num), (r
        - p0).dot(num)) / dp.dist2();
}
```

2.10 Angle class

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage:
vector<Angle> v = {w[0], w[0].t360() ...};
// sorted int j = 0; rep(i,0,n) { while (v[j]
< v[i].t180()) ++j; } // sweeps j such that
(j-i) represents the number of positively
oriented triangles with vertices at 0 and i

```
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t = 0) : x(x),
        y(y), t(t) {}
    Angle operator-(Angle b) const {
        return {x - b.x, y - b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t +
        (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t
        + half()}; }
    Angle t360() const { return {x, y, t +
        1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also
    // compare distances
    return make_tuple(a.t, a.half(), a.y *
        (11)b.x) <
        make_tuple(b.t, b.half(), a.x *
        (11)b.y);
}
// Given two points, this calculates the
// smallest angle between
// them, i.e., the angle that covers the
// defined line segment.
pair<Angle, Angle> segmentAngles(Angle a,
    Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ? make_pair(a, b)
        : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { //
    point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { //
    angle b - angle a
    int tu = b.t - a.t;
    a.t = b.t;
    return {a.x * b.x + a.y * b.y, a.x * b
        .y - a.y * b.x, tu - (b < a)};
}
```

2.11 Circle Tangents

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0. $O(n)$

```
template <class P>
vector<pair<P, P>> tangents(P c1, double
    r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(),
        h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2)
            * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v *
            r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

2.12 Minimum Enclosing Circle

Description: Computes the minimum circle that encloses a set of points. $O(n)$

```
pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i, 0, sz(ps)) if ((o - ps[i]).dist()
        > r * EPS) {
        o = ps[i], r = 0;
        rep(j, 0, i) if ((o - ps[j]).dist()
            > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k, 0, j) if ((o - ps[k]).dist()
                > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

2.13 Inside Polygon

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow. $O(n)$

Usage:
vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool
in = inPolygon(v, P{3, 3}, false);

```
template <class P>
bool inPolygon(vector<P>& p, P a, bool
    strict = true) {
    int cnt = 0, n = p.size();
    rep(i, 0, n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !
            strict;
        // or: if (segDist(p[i], q, a) <=
        // eps) return !strict;
        cnt ^= ((a.y < p[i].y) - (a.y < q.y)
            ) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

2.14 Polygon Area

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
template <class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i, 0, sz(v) - 1) a += v[i].cross(v
        [i + 1]);
    return a;
}
```

2.15 Polygon Center

Description: Returns the center of mass for a polygon. Time: $O(n)$

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0);
    double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v);
        j = i++) {
        res = res + (v[i] + v[j]) * v[j].
            cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

2.16 Convex Hull

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. $O(n \log n)$

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    int n = pts.size();
    if (n <= 1) return pts;
    sort(all(pts));
    vector<P> h(n + 1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t,
        reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t - 2].
                cross(h[t - 1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t
        == 2 && h[0] == h[1])};
}
```

2.17 Hull Diameter

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points). $O(n)$

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S
        [0]}});
    rep(i, 0, j) for (; j = (j + 1) % n)
        {
            res = max(res, {(S[i] - S[j]).dist2()
                , {S[i], S[j]}});
            if ((S[(j + 1) % n] - S[j]).cross(S[
                i + 1] - S[i]) >= 0) break;
        }
    return res.second;
}
```

2.18 Point Inside Hull

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included. $O(\log N)$

```
typedef Point<ll> P;
bool inHull(vector<P>& l, P p, bool
    strict = true) {
    int n = l.size(), a = 1, b = n - 1, r
        = !strict;
    if (n < 3) return r && onSegment(l[0],
        l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap
        (a, b);
    if (sideOf(l[0], l[a], p) >= r ||
        sideOf(l[0], l[b], p) <= -r) return
        false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a)
            = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

2.19 Line Hull Intersection

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. `lineHull(line, poly)` returns a pair describing the intersection of a line with the polygon:

- $(-1, -1)$ if no collision,
- $(i, -1)$ if touching the corner i ,
- (i, i) if along side $(i, i+1)$,
- (i, j) if crossing sides $(i, i+1)$ and $(j, j+1)$.

In the last case, if a corner i is crossed, this is treated as happening on side $(i, i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line. $O(\log N)$

```
#define cmp(i, j) sgn(dir.perp().cross(
    poly[(i) % n] - poly[(j) % n]))
#define extr(i) cmp(i + 1, i) >= 0 &&
    cmp(i, i - 1 + n) < 0
template <class P>
int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m
            + 1, m);
        (ls < ms || (ls == ms && ls == cmp(
            lo, m)) ? hi : lo) = m;
    }
    return lo;
}
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<
    P>& poly) {
    int endA = extrVertex(poly, (a - b).
        perp());
    int endB = extrVertex(poly, (b - a).
        perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(
            poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 :
                n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi)
                = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0],
        -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly)
            + 1) % sz(poly)) {
            case 0:
                return {res[0], res[0]};
            case 2:
                return {res[1], res[1]};
        }
    return res;
}
```

2.20 Closest Pair

Description: Finds the closest pair of points. $O(n \log n)$

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y
        < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P
        (), P()}};
    int j = 0;
    for (P p : v) {
        P d{1 + (ll)sqrt(ret.first), 0};
        while (v[j].y <= p.y - d.x) S.erase(
            v[j++]);
```

```
        auto lo = S.lower_bound(p - d), hi =
            S.upper_bound(p + d);
        for (; lo != hi; ++lo) ret = min(ret
            , {( *lo - p).dist2(), { *lo, p}});
        S.insert(p);
    }
    return ret.second;
}
```

2.21 Polyhedron Volume

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

```
template <class V, class L>
double signedPolyVolume(const V& p,
    const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].
        cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

2.22 Point 3D

Description: lass to handle points in 3D space. T can be e.g. double or long long.

```
template <class T>
struct Point3D {
    typedef Point3D P;
    typedef const P& R;
    T x, y, z;
    explicit Point3D(T x = 0, T y = 0, T z
        = 0) : x(x), y(y), z(z) {}
    bool operator<(R p) const { return tie
        (x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(R p) const { return
        tie(x, y, z) == tie(p.x, p.y, p.z);
    }
    P operator+(R p) const { return P(x +
        p.x, y + p.y, z + p.z); }
    P operator-(R p) const { return P(x -
        p.x, y - p.y, z - p.z); }
    P operator*(T d) const { return P(x *
        d, y * d, z * d); }
    P operator/(T d) const { return P(x /
        d, y / d, z / d); }
    T dot(R p) const { return x * p.x + y
        * p.y + z * p.z; }
    P cross(R p) const {
        return P(y * p.z - z * p.y, z * p.x
            - x * p.z, x * p.y - y * p.x);
    }
    T dist2() const { return x * x + y * y
        + z * z; }
    double dist() const { return sqrt((
        double)dist2()); }
    // Azimuthal angle (longitude) to x-
    // axis in interval [-pi, pi]
    double phi() const { return atan2(y, x
        ); }
    // Zenith angle (latitude) to the z-
    // axis in interval [0, pi]
    double theta() const { return atan2(
        sqrt(x * x + y * y), z); }
    P unit() const { return *this / (T)
        dist(); } // makes dist()=1
    // returns unit vector normal to *this
    // and p
    P normal(P p) const { return cross(p).
        unit(); }
    // returns point rotated 'angle'
    // radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle
        );
        P u = axis.unit();
        return u * dot(u) * (1 - c) + (*this
            ) * c - cross(u) * s;
    }
};
```

2.23 3D Convex Hull

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards. Time: $O(n^2)$

```
struct PR {
    void ins(int x) { (a == -1 ? a : b) =
        x; }
    void rem(int x) { (a == x ? a : b) =
        -1; }
    int cnt() { return (a != -1) + (b !=
        -1); }
    int a, b;
};
struct F {
    P3 q;
    int a, b, c;
};
vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR
        >(sz(A), {-1, -1}));
#define E(x, y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int
        l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A
            [i]));
        if (q.dot(A[l]) > q.dot(A[i])) q = q
            * -1;
        F f{q, i, j, k};
        E(a, b).ins(k);
        E(a, c).ins(j);
        E(b, c).ins(i);
        FS.push_back(f);
    };
    rep(i, 0, 4) rep(j, i + 1, 4) rep(k, j
        + 1, 4) mf(i, j, k, 6 - i - j - k)
        ;
    rep(i, 4, sz(A)) {
        rep(j, 0, sz(FS)) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a
                ])) {
                E(a, b).rem(f.c);
                E(a, c).rem(f.b);
                E(b, c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j, 0, nw) {
            F f = FS[j];
#define C(a, b, c) \
            if (E(a, b).cnt() != 2) mf(f.a, f.b, i
                , f.c);
            C(a, b, c);
            C(a, c, b);
            C(b, c, a);
        }
    }
    for (F& it : FS)
        if ((A[it.b] - A[it.a]).cross(A[it.c
            ] - A[it.a]).dot(it.q) <= 0)
            swap(it.c, it.b);
    return FS;
};
```

2.24 Spherical Distance

Description: Returns the shortest distance on the sphere with radius `radius` between the points with azimuthal angles (longitude) `f1` (ϕ_1) and `f2` (ϕ_2) from x axis and zenith angles (latitude) `t1` (θ_1) and `t2` (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. `dx*radius` is then the difference between the two points in the x direction and `d*radius` is the total distance between the points.

```
double sphericalDistance(double f1,
    double t1, double f2, double t2,
    double radius)
{
    double dx = sin(t2) * cos(f2) - sin(t1
        ) * cos(f1);
    double dy = sin(t2) * sin(f2) - sin(t1
        ) * sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx * dx + dy * dy + dz
        * dz);
```

```
    return radius * 2 * asin(d / 2);  
}
```